```
flatten = (arrays) ->
  total = []
  for array in arrays
    total = total.concat array
  total
```

The concat method returns a new array of the two arrays combined.

Now you have *three* similar functions. That's two too many. Instead of having different variants of the same function, if only you could avoid this repetition and have the common parts of the three functions in one place and the differences in another. Indeed you can! Extract the common parts of `add`, `multiply`, and `flatten` into a single function that takes another function as an argument. The function passed in as the argument will perform the parts that are different. Now, instead of using specific `add`, `multiply`, and `flatten` functions, you've abstracted away the common parts and made an `accumulate` function:

```
accumulate = (initial, numbers, accumulator) ->
  total = initial or 0
  for number in numbers
    total = accumulator total, number
  total
```

Add initial value and accumulator arguments.

The total needs to be initialized; use 0 as the default.

The accumulator function is invoked inside the comprehension.

Inside `accumulate`, instead of there being a primitive operator such as `+` or `*`, you see the name of the argument, `accumulator`. Use the new `accumulate` function to sum an array:

```
sum = (acc, current) -> acc + current
accumulate(0, [5,5,5], sum)
# 15
```

Define a sum accumulator.

Invoke accumulate with initial of 0, an array of numbers, and the sum function.

Now use the `accumulate` function to flatten an array:

```
flatten = (acc,current) -> acc.concat current
accumulate([], [[1,3],[2,8]], flatten)
# [1,3,2,8]
```

Try to visualize this `accumulate` function to understand how it works.

Visualizing accumulate